

# Explain DP Path Cost Code

This is **LeetCode 2304 – Minimum Path Cost in a Grid**. The important part is understanding what `dp[i][j]` means. Once that is clear, the recurrence and code become straightforward.

## 1. What is the problem asking?

You are given two things:

`grid`

For example:

```
text
grid =
[
  [5, 3],
  [4, 0],
  [2, 1]
]
```

You start from **any cell in the first row**.

From a cell in row `i`, column `k`, you can move to **any column** in the next row.

So:

```
text
row i                row i+1

grid[i][k]  -----> grid[i+1][0]
              -----> grid[i+1][1]
              -----> grid[i+1][2]
              ...
```

But moving has an additional cost.

That cost is stored in:

```
C++
moveCost[value][nextColumn]
```

So if your current cell contains value `5` and you move to column `2` in the next row:

```
C++
moveCost[5][2]
```

is the movement cost.

## 2. What exactly are we minimizing?

Suppose we have:

```
text
grid:

      col0 col1 col2
row0  5   3   2
row1  4   0   6
row2  1   7   3
```

Suppose we choose:

```
text
5 → 0 → 3
```

The total cost is:

```
text
5           // starting cell
+ moveCost[5][1] // 5 → column 1
+ 0         // second cell
+ moveCost[0][2] // 0 → column 2
+ 3         // final cell
```

We need the **minimum possible total cost** from the first row to the last row.

### 3. The key observation

The movement from the previous row to the current row can come from **any column**.

For example, to reach:

```
text
grid[i][j]
```

we could have come from:

```
text
grid[i-1][0]
grid[i-1][1]
grid[i-1][2]
...
grid[i-1][m-1]
```

So we simply ask:

```
Which previous column gives me the cheapest path to grid[i][j]?
```

That is exactly what this loop does:

```
C++
for(int k = 0; k < m; k++) {
    int val = grid[i-1][k];
    int cost = moveCost[val][j];

    minval = min(minval, dp[i-1][k] + cost);
}
```

### 4. What does `dp[i][j]` mean?

This is the **most important concept**.

We define:

```
text
dp[i][j] =
minimum total cost required to reach cell (i, j)
```

So:

```
text
dp[i][j]
```

already contains:

- all grid values collected before reaching (i,j)
- all movement costs paid before reaching (i,j)
- the current cell's value

For example:

```
text
dp[2][1]
```

means:

```
Minimum cost to start somewhere in row 0 and eventually reach row 2, column 1.
```

### 5. Base case

We can start from **any cell in the first row**.

Therefore:

```
C++
vector<vector<int>> dp = grid;
```

initializes:

```
text
dp[0][0] = grid[0][0]
dp[0][1] = grid[0][1]
dp[0][2] = grid[0][2]
...
```

Why?

Because if we start at  $(0, j)$ , there is no movement cost yet.

So:

```
text
dp[0][j] = grid[0][j]
```

This is our base case.

---

## 6. Deriving the recurrence relation

Now suppose we want:

```
text
dp[i][j]
```

We can reach  $(i, j)$  from any column  $k$  in the previous row.

The previous cell is:

```
text
(i-1, k)
```

The cost of reaching that previous cell is:

```
text
dp[i-1][k]
```

Then we move from:

```
text
grid[i-1][k]
```

to:

```
text
grid[i][j]
```

The movement cost is:

```
C++
moveCost[grid[i-1][k]][j]
```

Then we add the value of the current cell:

```
C++
grid[i][j]
```

Therefore:

```
text
dp[i][j]
=
grid[i][j]
+
minimum over all k:
    dp[i-1][k] + moveCost[grid[i-1][k]][j]
```

### Final recurrence

```
\[
\boxed{
dp[i][j] = grid[i][j] +
\min_{k=0}^{m-1}
\left(dp[i-1][k] + moveCost[grid[i-1][k]][j]\right)
}
\]
```

This is the entire DP.

---

## 7. Why do we use `grid[i-1][k]` inside `moveCost`?

This line often confuses people:

```
C++
int val = grid[i-1][k];
int cost = moveCost[val][j];
```

Notice:

```
C++
moveCost[val][j]
```

not:

```
C++
moveCost[grid[i][j]][j]
```

Why?

Because the movement cost depends on the **value of the cell we are leaving**.

Suppose:

```
text
grid[i-1][k] = 7
```

and we're moving to column  $j = 2$ .

Then:

```
C++
moveCost[7][2]
```

is the cost.

The destination's value doesn't determine the movement cost.

---

## 8. Let's understand the code line by line

Your code:

```
C++
class Solution {
public:
    int minPathCost(vector<vector<int>>& grid, vector<vector<int>>& moveCost) {
```

The function receives:

```
C++
grid
moveCost
```

and returns the minimum path cost.

---

### Get dimensions

```
C++
int n = grid.size();
int m = grid[0].size();
```

$n$  = number of rows.

$m$  = number of columns.

---

### Create DP table

```
C++
vector<vector<int>> dp = grid;
```

Initially:

```
text
dp = grid
```

This conveniently gives us the base case:

```
text
dp[0][j] = grid[0][j]
```

---

## 9. Process every row after the first

```
C++
for(int i = 1; i < n; i++) {
```

We start from:

```
text
```

```
i = 1
```

because row 0 is already initialized.

Then:

```
C++
```

```
for(int j = 0; j < m; j++) {
```

We calculate the answer for every cell in the current row.

So we're calculating:

```
text
```

```
dp[i][0]  
dp[i][1]  
dp[i][2]  
...  
dp[i][m-1]
```

## 10. Find the best previous cell

```
C++
```

```
int minval = INT_MAX;
```

We initially assume the best previous path is infinitely expensive.

Then:

```
C++
```

```
for(int k = 0; k < m; k++) {
```

Try every possible previous column.

So:

```
text
```

```
current cell = (i,j)
```

```
possible previous cells:
```

```
(i-1,0)  
(i-1,1)  
(i-1,2)  
...  
(i-1,m-1)
```

### Get previous cell's value

```
C++
```

```
int val = grid[i-1][k];
```

Suppose:

```
text
```

```
grid[i-1][k] = 5
```

Then:

```
C++
```

```
val = 5
```

### Get movement cost

```
C++
```

```
int cost = moveCost[val][j];
```

If:

```
text
```

```
val = 5  
j = 2
```

then:

```
C++  
cost = moveCost[5][2];
```

## Calculate candidate path

```
C++  
dp[i-1][k] + cost
```

This means:

```
text  
minimum cost to reach previous cell  
+  
cost of moving to current column
```

We haven't added the current cell's value yet.

## Keep the minimum

```
C++  
minval = min(minval, dp[i-1][k] + cost);
```

After checking every  $k$ :

```
text  
minval
```

contains the cheapest way to arrive at column  $j$  from the previous row.

## 11. Add the current cell

Once we know the cheapest way to get here:

```
C++  
dp[i][j] = grid[i][j] + minval;
```

Why add `grid[i][j]`?

Because the path cost includes the value of every cell visited.

So:

```
text  
DP to previous cell  
+  
movement cost  
+  
current cell value
```

Exactly:

```
text  
dp[i][j] =  
min(previous DP + movement cost)  
+  
current grid value
```

## 12. Final answer

After processing all rows:

```
C++  
int ans = INT_MAX;
```

Then:

```
C++  
for(int j = 0; j < m; j++) {  
    ans = min(ans, dp[n-1][j]);  
}
```

Why the last row?

Because the path can finish at **any column** of the last row.

Therefore:

```

text
answer =
min(
    dp[last row][0],
    dp[last row][1],
    ...
    dp[last row][m-1]
)

```

Then:

```

C++
return ans;

```

## 13. Small dry run

Let's use:

```

text
grid =
[
    [5, 3],
    [4, 0],
    [2, 1]
]

```

Suppose:

```

text
moveCost =
[
    ...
]

```

Let's focus on one calculation.

Suppose we're calculating:

```

text
dp[1][1]

```

That means:

Minimum cost to reach row 1, column 1.

Possible previous cells:

```

text
    row 0
    5    3
    ↓    ↓
    ?    ?
    row 1

```

There are two possibilities.

### From column 0

```

text
dp[0][0] + moveCost[5][1]

```

### From column 1

```

text
dp[0][1] + moveCost[3][1]

```

Therefore:

```

text
dp[1][1] =
grid[1][1]
+
min(
    dp[0][0] + moveCost[5][1],
    dp[0][1] + moveCost[3][1]
)

```

Since:

```
text
grid[1][1] = 0
```

we get:

```
text
dp[1][1] =
0 + min(...)
```

That's exactly what your nested loops calculate.

## 14. Think of the DP as a graph

This makes the problem even easier to understand.

Every grid cell is a node.

For example:

```
text
      5      3      2
    /|\ /|\ /|\
   / | \ / | \ / | \
  4 0 6 4 0 6 4 0 6
   \ | / \ | / \ | /
    ↓   ↓   ↓
   ...
```

You can move from every cell in one row to **every cell in the next row**.

Each transition has a weight:

```
text
moveCost[currentValue][destinationColumn]
```

And every destination node contributes:

```
text
grid[i][j]
```

So the problem is essentially:

Find the minimum-cost path through this layered graph.

Because movement only goes **downward**, we can solve it with DP.

## 15. Why is this DP and not greedy?

This is important.

You might think:

"At every row, just choose the cheapest next cell."

That doesn't work.

Why?

Because the movement cost depends on the **current value**.

A cell that looks cheap right now might have an expensive movement cost to the next row.

For example:

```
text
Current choices:

A → cheap cell
B → expensive cell
```

But maybe:

```
text
A → next row = 100
B → next row = 5
```

So choosing locally based on the current cell isn't necessarily optimal.

DP considers **all possible previous states** and stores the best result for each state.

## 16. Complexity

There are:

```
text
n × m
```

DP states.

For every state  $(i, j)$ , we check all  $m$  previous columns.

Therefore:

```
\[
O(n \times m \times m)
\]
```

or:

```
\[
\boxed{O(nm^2)}
\]
```

Space:

```
C++
vector<vector<int>> dp
```

contains  $n \times m$  elements.

Therefore:

```
\[
\boxed{O(nm)}
\]
```

## 17. The whole DP pattern

This problem follows a very common pattern:

### State

```
text
dp[i][j]
```

= minimum cost to reach cell  $(i, j)$ .

### Base case

```
text
dp[0][j] = grid[0][j]
```

### Transition

```
text
dp[i][j] =
grid[i][j] +
min(
    dp[i-1][k] + moveCost[grid[i-1][k]][j]
)
```

for every:

```
text
k = 0 ... m-1
```

### Answer

```
text
min(dp[n-1][j])
```

## The code in a more readable form

```
C++
class Solution {
public:
    int minPathCost(vector<vector<int>>& grid, vector<vector<int>>& moveCost) {
        int n = grid.size();
        int m = grid[0].size();

        // dp[i][j] = minimum cost to reach (i, j)
```

```
vector<vector<int>>> dp = grid;

// First row is already initialized:
// dp[0][j] = grid[0][j]

for (int i = 1; i < n; i++) {

    for (int j = 0; j < m; j++) {

        int minval = INT_MAX;

        // Try every cell in previous row
        for (int k = 0; k < m; k++) {

            // Value of the cell we are leaving
            int val = grid[i - 1][k];

            // Cost to move from that value
            // to column j
            int cost = moveCost[val][j];

            // Cost of reaching previous cell
            // + cost of moving to current column
            minval = min(minval, dp[i - 1][k] + cost);
        }

        // Add current cell's value
        dp[i][j] = grid[i][j] + minval;
    }
}

// We can finish at any column in last row
int ans = INT_MAX;

for (int j = 0; j < m; j++) {
    ans = min(ans, dp[n - 1][j]);
}

return ans;
}
```

## The one line you should remember

The entire problem boils down to:

```
text
dp[i][j] = current cell
           + cheapest way to arrive here
```

and

```

text
cheapest way to arrive here
=
min over all previous columns (
    previous DP
    + movement cost
)

```

So the mental template is:

text

previous row

|     |     |     |
|-----|-----|-----|
| k=0 | k=1 | k=2 |
|-----|-----|-----|

movement costs

current  
(i,j)

```
dp[i][j] = grid[i][j] + minimum(all possible previous paths)
```

That's why the **three nested loops** are necessary: current row → current column → every possible previous column.